

Malware Detection Using Byteplot Image Representations and Deep CNN Architectures

Habeeb Dashti · Shehzar Aurangzeb Abbasi · Usaib Khan
· Yaser Esmaeili Salehani · Zahra Ebrahimi

Department of Applied Computer Science, Concordia
University, Montreal, Quebec, Canada

Abstract— The rapid evolution of malware, often generated through minor code variations by cybercriminal groups, poses a persistent challenge to traditional detection mechanisms such as static signature and behavior-based analysis. To address the limitations of handcrafted feature extraction and high computational overhead in classical machine learning methods, we propose a deep learning-based malware detection system that leverages image-based representations of Portable Executable (PE) files. These byteplot images are analyzed using state-of-the-art Convolutional Neural Network (CNN) architectures ResNet-18, EfficientNet-B0, and DenseNet-121 selected for their balance between accuracy and efficiency. Our system undergoes rigorous preprocessing, including merging similar malware families, enhanced data augmentation, image resizing, class balancing through augmentation, normalization, and optimized data partitioning, to enhance generalization. Experiments conducted on three malware image datasets of varying scales demonstrate strong classification performance, with test accuracies reaching up to 96.70%. To mitigate overfitting and further enhance detection accuracy, we employed advanced regularization techniques and ensemble learning strategies, including stacking ensembles with Random Forest and Gradient Boosting, which achieved up to 100.0% test accuracy. Visualization methods such as Grad-CAM were used to interpret model decisions, reinforcing model transparency. Overall, the proposed approach is computationally efficient, scalable, and robust against class imbalance and obfuscation, offering a reliable framework for next-generation malware detection.

Index Terms— Malware Detection, Convolutional Neural Networks, Byteplot Images, Portable Executable Files, Ensemble Learning.

I. INTRODUCTION

The exponential growth in cybercrime has made malware one of the most pressing threats in today’s digital landscape. With millions of new malware samples emerging annually, the adaptability and sophistication of malicious software continue to outpace traditional detection systems. Conventional methods such as static signature analysis and heuristic-based tools, while useful against known threats, struggle to detect novel or obfuscated variants due to their reliance on predefined rules and handcrafted features [1]. Dynamic analysis offers deeper behavioral insights but is resource-intensive and susceptible to evasion techniques employed by malware aware of virtualized sandbox environments [2]. This evolving cat-and-mouse game between malware authors and security systems necessitates more resilient, scalable, and intelligent detection frameworks.

Recent advances in deep learning, particularly in computer vision, have introduced a promising alternative: transforming binary malware files into image representations and treating malware detection as a visual classification task. By converting Portable Executable (PE) files into grayscale byteplot images, it becomes possible to capture structural patterns and textures that are otherwise invisible to traditional techniques [3]. These images, when analyzed through Convolutional Neural Networks (CNNs), enable the model to learn discriminative features across malware families without manual feature engineering. Moreover, leveraging pretrained CNN architectures allows for efficient transfer learning, enabling robust detection even in data-scarce environments [4].

In this paper, we present a deep learning-based malware detection system that employs three well-established CNN models ResNet-18, EfficientNet-B0, and DenseNet-121 to classify malware from byteplot images. Our approach includes rigorous preprocessing techniques such as resizing, normalization, data augmentation, and class balancing to ensure reliable training across diverse datasets. We evaluate the models on three publicly available malware image datasets of varying complexity and scale, using standard classification metrics such as precision, recall, and F1-score to assess performance. Furthermore, to address overfitting and boost generalization, we integrate ensemble learning strategies like stacking with Random Forest and Gradient Boosting. Our experimental results demonstrate that this image-based detection system is both accurate and resource-efficient, offering a scalable solution to modern malware detection challenges [5].

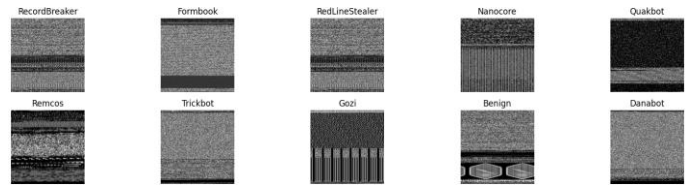


Figure 1. A visual overview of byteplot images from different malware classes, illustrating structural similarities within families and differences across them.

II. RELATED WORK

Traditional malware detection techniques are broadly classified into static, dynamic, and hybrid analysis methods. Static analysis inspects software code without execution, extracting features such as byte sequences, opcode patterns, and control flow graphs [6]. While it is efficient, static detection struggles with polymorphic and obfuscated malware,

which modify code structures to evade signature matching [7]. Dynamic analysis, on the other hand, observes malware behavior during execution in controlled environments, capturing features like system calls, registry modifications, and network traffic [8]. Though more robust to code obfuscation, dynamic methods are computationally expensive and vulnerable to anti-analysis techniques used by malware that detect sandbox environments [4]. Hybrid approaches attempt to combine the strengths of both but often introduce complexity and require significant domain expertise [9].

In response to these limitations, recent research has increasingly adopted machine learning and deep learning methods to automate and enhance malware detection. Classical ML models such as Support Vector Machines (SVM), Random Forests, and Decision Trees have been trained on hand-engineered features extracted from PE files or behavior logs [6]. However, these models are sensitive to feature selection and typically lack the scalability to handle large, diverse malware datasets.

A promising alternative is the transformation of malware binaries into images, enabling the use of computer vision techniques for classification. Nataraj et al. pioneered this concept by converting binaries into grayscale images and applying GIST texture features for KNN-based classification [10]. Subsequent studies have explored various image representation techniques ranging from raw byte-level images to entropy and opcode-based visualization each capturing different aspects of malware structure [11], [12]. Convolutional Neural Networks (CNNs), with their ability to learn hierarchical features directly from raw pixel data, have become the architecture of choice for these tasks. Models like ResNet, DenseNet, and EfficientNet have demonstrated strong performance across different datasets, often outperforming classical methods in both accuracy and robustness [13], [14].

Transfer learning has further accelerated development by allowing pre-trained CNNs to adapt to malware datasets with limited samples, reducing training time and improving generalization [15]. Ensemble methods, including stacking, boosting, and bagging, have also been employed to mitigate overfitting and enhance classification stability [16]. While most studies have focused on single-platform datasets (e.g., Windows executables or Android APKs), few have addressed cross-platform generalizability or benchmarked multiple CNN architectures under a unified experimental setup. Moreover, research on ensemble-based malware detectors using image representations remains limited, particularly in the context of evaluating trade-offs between performance and computational cost.

This study builds upon the vision-based paradigm, employing three CNN models ResNet-18, EfficientNet-B0, and DenseNet-121 on PE byteplot images for malware classification. Unlike previous work, our system integrates a comprehensive preprocessing pipeline, explores ensemble stacking for generalization, and benchmarks performance

across multiple datasets. These contributions position our approach as a scalable and adaptable solution for modern malware detection.

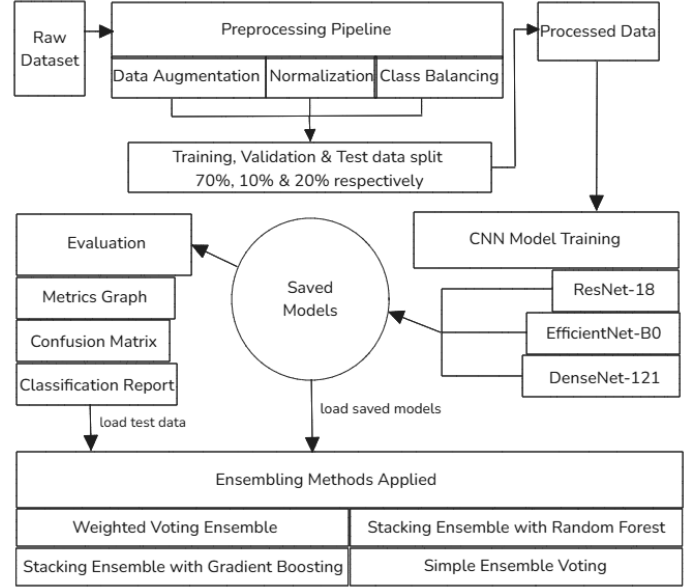


Figure 2. The architecture of the proposed CNN-based malware detection developed in this work, including preprocessing, training, and ensemble evaluation.

III. NOVEL CONTRIBUTIONS

This work introduces a malware detection approach that integrates multiple convolutional neural networks (CNNs) with ensemble learning techniques to improve accuracy and generalization. Unlike previous studies that typically use a single CNN model or rely solely on classical machine learning algorithms [5] we evaluated several CNN architectures DenseNet-121, EfficientNet-B0, and ResNet-18 each offering distinct representational capabilities. These models were then combined using ensemble methods, including softmax averaging, weighted voting, and stacking ensembles with Random Forest and Gradient Boosting as meta-learners. This layered design allowed the system to capture diverse features and reduce overfitting, with the stacking ensemble using Random Forest achieving the best results. To remain scientifically cautious, we report that our approach achieved high accuracy on selected datasets under constrained conditions. Future work will focus on evaluating the model's performance on broader, more varied malware families to ensure robust generalization. Furthermore, to improve interpretability, we used Grad-CAM visualizations to show which parts of the malware images most influenced classification decisions—an essential step in making deep learning models more transparent for cybersecurity analysts

IV. METHODOLOGY

This section outlines the full pipeline of our proposed malware detection system using deep learning-based image classification. We describe the datasets used, preprocessing

methods applied, CNN architectures implemented, and the overall training and evaluation process.

A. Dataset Description

To train and evaluate our system, we used three malware image datasets that represent different classification challenges in terms of scale, class balance, and label granularity:

1. **Blended Malware Image Dataset** [17]: Contains 9,868 PNG images across 31 malware families. Each image has a resolution of 224×224 pixels.
2. **Malware Images Dataset** [18]: Comprises 125,004 grayscale images across 17 classes. It exhibits class imbalance, requiring augmentation for effective training.
3. **Malware-Byteplot Images Dataset** [19]: Features 60,337 byteplot images labeled as either “Benign” or “Malicious” for binary classification.

We chose byteplot images because they provide a **visual representation** of byte sequences in executable files, capturing important spatial patterns and structural features that enhance the model's ability to distinguish between malware and benign files. This approach contrasts with other possible representations, such as **grayscale binaries** or **entropy-based images**, which may not capture the same level of detailed byte-level information. Furthermore, byteplot images were readily available in the datasets we selected, making them a practical and effective choice for our classification task.

B. Image Preprocessing

To ensure uniformity and enhance learning, we applied a structured preprocessing pipeline across all datasets:

1. **Image Resizing:** All images are resized to 224×224 pixels to ensure input consistency for CNNs with fixed kernel sizes [11].
2. **Normalization:** Pixel values are scaled from [0, 255] to [0, 1] using the transformation:

$$I_{norm} = \frac{I}{255}$$

This helps reduce training instability and speeds up convergence by ensuring consistent gradient updates [20].

3. **Class Balancing:** For imbalanced datasets, we use:
 - a. **Undersampling** for overrepresented classes.

- b. **Oversampling** with Augmentation for minority classes using transformations such as horizontal flips, rotations, shifts, and brightness adjustments.

4. Data Testing:

- a. **Small/medium datasets:** 70% training, 10% validation, 20% testing.
- b. **Large dataset:** Predefined splits; we use 20% of validation for testing.

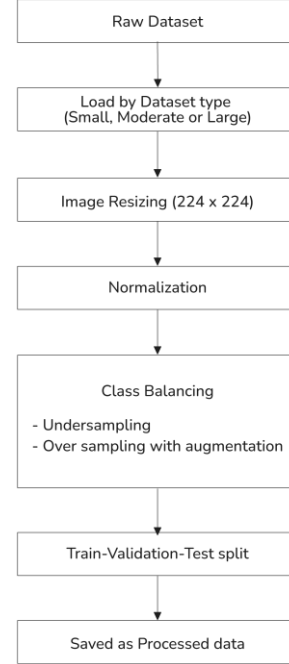


Figure 3. Illustration of the initial preprocessing steps and the distribution of malware classes within the training, validation, and testing datasets.

While the initial preprocessing pipeline yielded robust performance on both small and large datasets, significant overfitting was observed in the moderate dataset. For instance, ResNet18 achieved a training accuracy of 93.67%, while validation and test accuracies dropped to 64.46% and 63.36%, respectively. Comparable overfitting trends were observed with DenseNet121 and EfficientNetB0. To address this issue, the preprocessing pipeline was further enhanced with the following measures:

5. **Merged Class:** Based on observations from the confusion matrix and manual inspection, it was found that the *RecordBreaker* and *RedLineStealer* classes contained visually identical images. To eliminate redundancy and improve classification clarity, these two classes were merged into a new unified class labeled *RecordLine*. This was implemented using a dictionary-based function that iterated through the dataset and reassigned class labels, ensuring

consistency and reducing ambiguity in the training data.

6. **Data Augmentation:** To improve model generalization, prevent overfitting, and balance class distribution, we applied a structured augmentation pipeline using the Albumentations library. The transformations were selected to simulate real-world variability in malware image patterns. Each transformation was applied with a specified probability p , as detailed below:

- a. **Random Brightness and Contrast Adjustment:** Applied with $p = 0.5$, this transformation adjusts image brightness β and contrast α modeled as:

$$I' = \alpha(I - \mu) + \mu + \beta$$

- b. **Gaussian Blur:** Applied with $p = 0.3$, this simulates smoothed images using a blur kernel $k \in [3, 7]$.
- c. **Gaussian Noise:** With $p = 0.4$, zero-centered noise $N(0, \sigma^2)$ is added to each pixel, where $\sigma^2 \in [10.0, 50.0]$, to simulate sensor noise or artifacts.
- d. **Elastic Transformation:** Applied at $p = 0.2$, this spatial deformation is defined by a displacement field controlled by parameters α , σ and α_{affine} , encouraging shape invariance.
- e. **Affine Transformation:** With $p = 0.5$, images are randomly:
- i. Scaled: $s \in [0.9, 1.1]$
 - ii. Translated: $t \in [-5\%, 5\%]$
 - iii. Rotated: $\theta \in [-10^\circ, 10^\circ]$
 - iv. Sheared: $\phi \in [-5^\circ, 5^\circ]$
- f. **Horizontal Flip:** Flips the image along the vertical axis with $p = 0.5$, introducing left-right invariance.
- g. **Vertical Flip:** Applied with $p = 0.2$, this handles malware families with symmetric structural patterns.
- h. **Coarse Dropout:** With $p = 0.4$, up to three random patches (each of size up to 32×32 pixels) are masked out to force the model to rely on distributed features rather than specific regions.

Additionally, all images were resized to a fixed resolution of 256×256 pixels prior to augmentation. While edge detection methods such as Canny, Sobel, and Hybrid filters were explored during initial experiments, empirical results showed that the original grayscale images consistently yielded better classification performance.

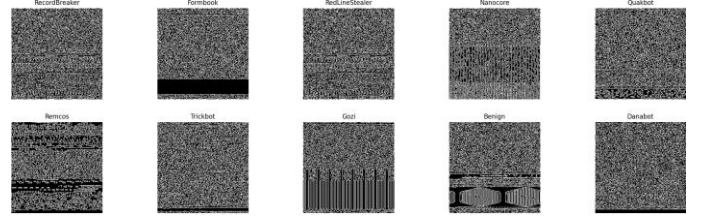


Figure 4. A visual overview of original byteplot images distorted by Canny edge detection.

Therefore, we opted to increase the image resolution from 224×224 to 256×256 to better preserve structural features in malware samples and enhance the effectiveness of convolutional kernels.

Training Accuracy	Validation Accuracy	Test Accuracy
93.67%	64.48%	63.36%

Figure 5. Overfitting in ResNet-18: High training accuracy with significantly lower validation and test accuracies on the moderate dataset.

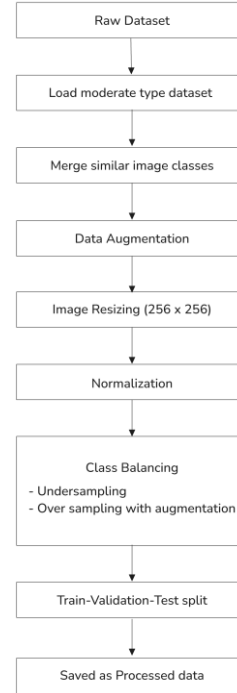


Figure 6. Finalized preprocessing pipeline incorporating class merging and enhanced data augmentation techniques.

C. CNN Architectures

We implemented three widely used CNN architectures ResNet-18, EfficientNet-B0, and DenseNet-121 for malware

classification. Each was selected for its balance of accuracy and computational efficiency.

1. **ResNet-18**: Utilizes residual connections to combat the vanishing gradient problem, allowing deeper network training [14].
2. **EfficientNet-B0**: Employs compound scaling of depth, width, and resolution for balanced accuracy and efficiency [13].
3. **DenseNet-121**: Features dense connectivity where each layer receives input from all preceding layers, enhancing feature reuse and gradient flow [21].

Each model is initialized with pretrained ImageNet weights. Final fully connected layers are replaced to match the number of output classes per dataset.

ResNet-18	EfficientNet-B0	DenseNet-121
Input 3x256x256	Input 3x256x256	Input 3x256x256
Conv 64	Conv 32	Conv 64
Residual Blocks Dropout $\rightarrow$ Linear	MB Conv Dropout $\rightarrow$ Linear	Dense Blocks Dropout $\rightarrow$ Linear
AdamW	SGD	SGD
Batch size 64	Batch size 32	Batch size 32

Figure 7. Representation of the architectural differences and parameter flow of the three CNN models used in our experiments.

D. Training Strategy

Our training strategy integrated several modern best practices for robust and interpretable CNN training.

1. **Data Pipeline**: Images were efficiently batched and shuffled using PyTorch DataLoader. Preprocessing steps included resizing, normalization, class balancing, and data augmentation.
2. **Model Initialization**: Final classification layers were adapted to match the number of output classes per dataset. Models were initialized with pretrained ImageNet weights, excluding the last layer.
3. **Optimization and Loss**:
 - a. **Optimizer**: SGD with momentum=0.9 and weight decay=5e-4
 - b. **Scheduler**: Cosine Annealing over 30 epochs
 - c. **Loss Functions**:
 - i. Categorical Cross-Entropy with label smoothing.
 - ii. Class-Balanced Cross-Entropy (CBCE) for imbalanced data:

$$L_{CBCE} = - \frac{1 - \beta}{1 - \beta^{n_y}} \log(y^{\wedge i})$$

where $\beta = \frac{l-1}{l}$ and n_y is the number of samples in class y

4. **Mixed Precision & Regularization**: Training was accelerated using PyTorch AMP via GradScaler. Regularization included:
 - a. Dropout layers (e.g., 0.7 in EfficientNet)
 - b. Label smoothing

5. Hyperparameters:

- a. Learning rate: 1×10^{-4}
 - b. Epochs: 10 (adjusted based on validation metrics)
 - c. Batch size: 32 (varied up to 64 depending on GPU availability)
6. **Evaluation Metrics**: All models were validated using accuracy, precision, recall, F1-score, and confusion matrix. Grad-CAM visualizations were used to interpret model decisions.

E. Interpretability

To improve the transparency of our models, we utilize **Grad-CAM** (Gradient-weighted Class Activation Mapping) visualizations, which highlight the regions of input byteplot images that contribute most to the model's predictions. These visualizations offer insight into how the models make decisions, revealing the class-discriminative regions that are most influential in distinguishing between malware and benign files.

We applied Grad-CAM to three different models: **DenseNet-121**, **EfficientNet-B0**, and **ResNet-18**, to provide a comprehensive understanding of their decision-making process. Each visualization shows the original byteplot image alongside the corresponding Grad-CAM heatmap, making it easier to see the regions that most influence the model's classification.

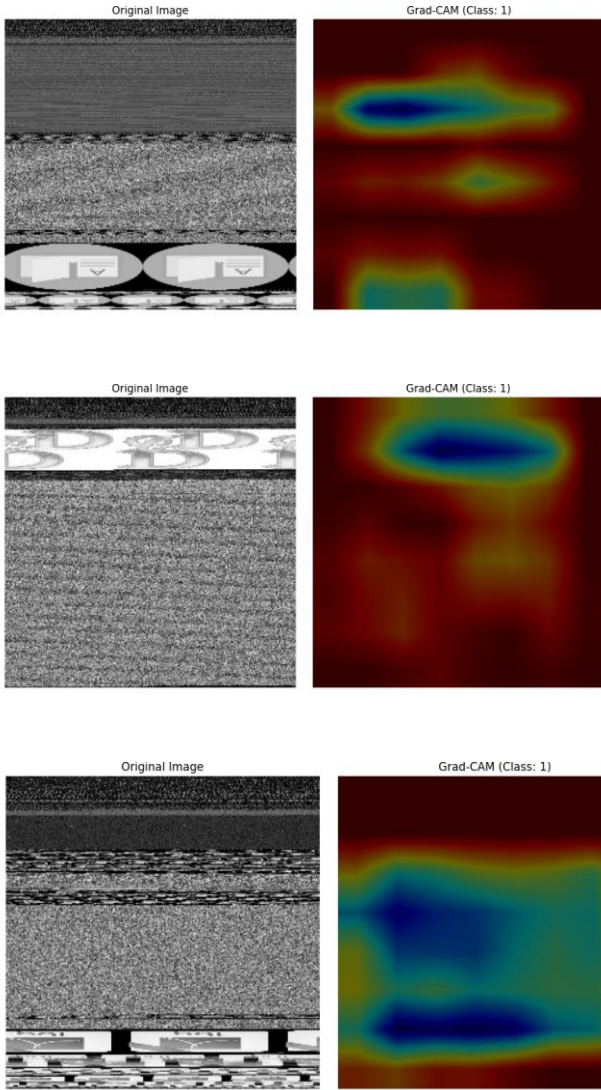


Figure 8. Grad-CAM visualizations of Class 1 samples from the Moderate dataset, highlighting the class-discriminative regions identified by three models: DenseNet-121, EfficientNet-B0, and ResNet-18 (from top to bottom). These visualizations reveal which regions of the input contribute most to model predictions, providing insight into the models' decision-making process

V. EXPERIMENTAL RESULTS

A. Experimental Setup

We evaluated three Convolutional Neural Network (CNN) architectures **ResNet-18**, **EfficientNet-B0**, and **DenseNet-121** across three malware image datasets (small, medium, and large). Each dataset was preprocessed by resizing images to 256×256 pixels, applying normalization, class balancing, and augmentation techniques. The data was split into 70% training, 10% validation, and 20% testing sets. All models were trained under identical hardware conditions using a fixed learning rate per model, batch size of 32 or 64 depending on model, and epochs ranging from 10 to 45 depending on convergence.

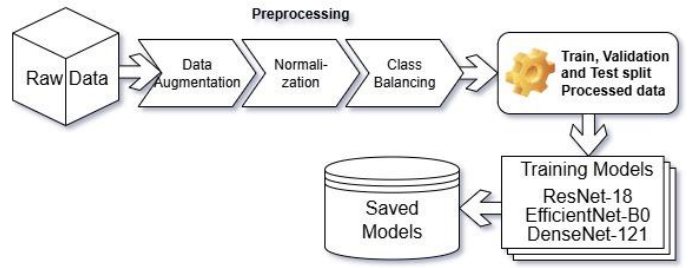


Figure 9. Pipeline diagram of dataset flow, its preprocessing and model training using CNN architectures.

B. Evaluation Metrics

We employed a comprehensive set of evaluation metrics to analyze model performance:

- **Accuracy:** Proportion of total correct predictions.
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall:** $\frac{TP}{TP+FN}$
- **F1-score:** Harmonic mean of precision and recall.
- **Confusion Matrix:** Visual assessment of correct vs. incorrect class predictions.

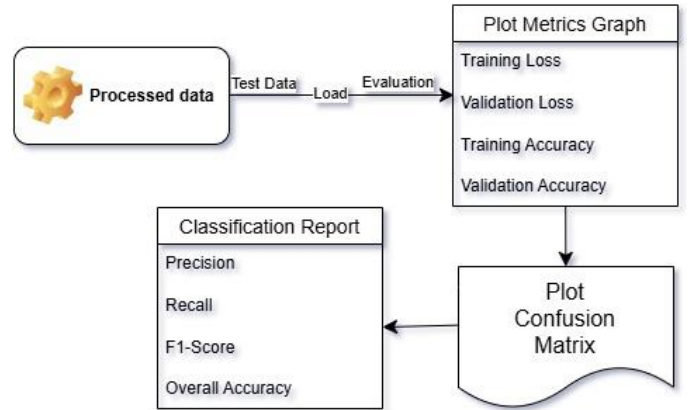


Figure 10. Pipeline diagram of evaluation process by loading test data.

C. Performance Results

Small Dataset: In the binary classification task (Benign vs. Malicious), all three CNNs performed consistently well. DenseNet-121 achieved 96.07%, EfficientNet-B0 97.57%, and ResNet-18 99.03% training accuracy with pretrained weights in 10 epochs. Validation accuracy closely followed, supported by Grad-CAM visualizations for model interpretability.

Model	Training Accuracy (%)	Validation Accuracy (%)	Test Accuracy (%)
DenseNet-121	96.07	94.95	95.35
EfficientNet-B0	97.57	95.69	95.93
ResNet-18	99.03	95.11	95.53

Figure 11. Accuracy performance of CNN models on the small dataset, trained for 10 epochs using pretrained weights.

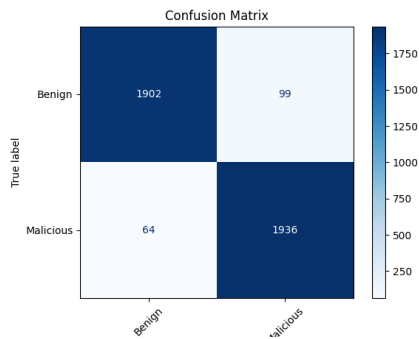


Figure 12. Confusion matrix of the EfficientNet-B0 model on the binary classification task. ResNet-18 and DenseNet-121 exhibited closely similar patterns of classification performance.



Figure 13. Training and validation accuracy and loss curves for EfficientNet-B0 showing stable convergence over 10 epochs for small dataset.

Medium Dataset: All three CNN models DenseNet-121, EfficientNet-B0 and ResNet-18 reached the highest training accuracies about 99%. Test accuracy reached 85–86% with pretrained weights at varying epochs ranging from 10 to 45.

Model	Training Accuracy (%)	Validation Accuracy (%)	Test Accuracy (%)	Epochs
DenseNet-121	99.82	85.60	84.68	45
EfficientNet-B0	99.76	86.88	85.21	30
ResNet-18	99.37	86.07	84.84	10

Figure 14. Accuracy performance of CNN models on the moderate dataset, trained using pretrained weights.

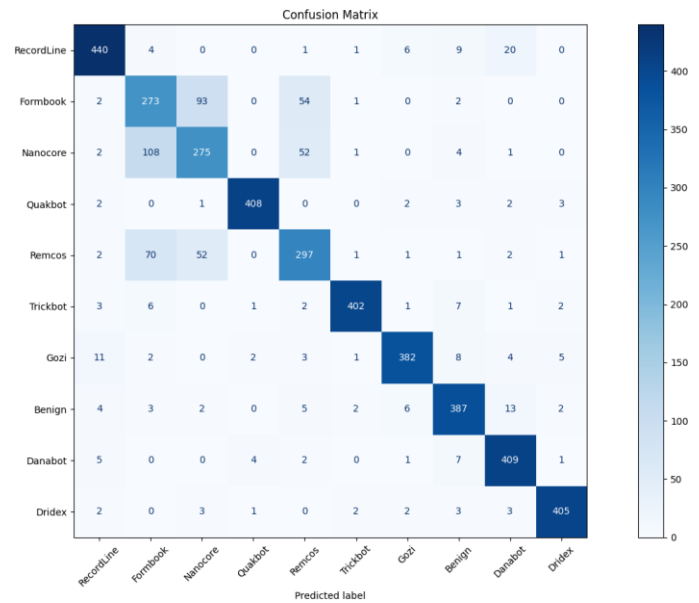


Figure 15. Confusion matrix of the DenseNet-121 model on the moderate dataset classification task. ResNet-18 and EfficientNet-B0 exhibited closely similar patterns of classification performance.



Figure 16. Training and validation accuracy and loss curves for DenseNet-121 showing stable convergence over 45 epochs with Cosine Annealing LR to reduce fluctuations and gain smoother convergence for moderate dataset.

Large Dataset: Training accuracy exceeded 97% for all models. EfficientNet-B0 and ResNet-18 both achieved test accuracy of ~96%, while DenseNet-121 closely followed with 94.53%. Some classes like "Nashta" remained challenging with lower precision and recall. All results were obtained over 10 epochs

Model	Training Accuracy (%)	Validation Accuracy (%)	Test Accuracy (%)
DenseNet-121	97.77	95.83	94.53
EfficientNet-B0	98.26	97.22	96.18
ResNet-18	99.18	97.57	96.70

Figure 17. Accuracy performance of CNN models on the large dataset, trained using pretrained weights at 10 epochs

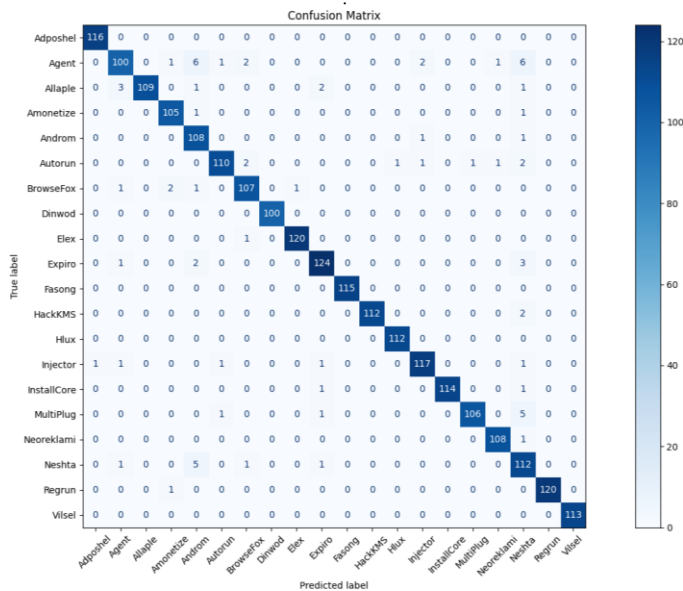


Figure 18. Confusion matrix of the ResNet-18 model on the large dataset classification task. DenseNet-121 and EfficientNet-B0 exhibited closely similar patterns of classification performance.

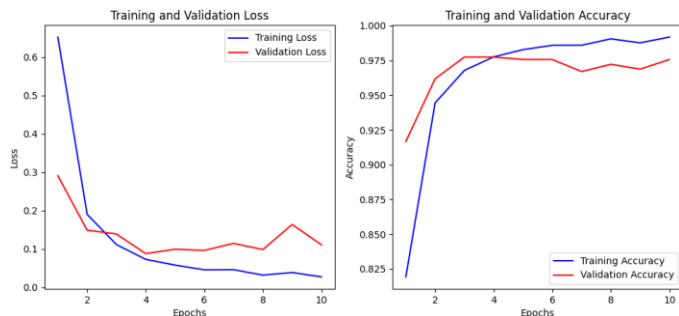


Figure 19. Training and validation accuracy and loss curves for ResNet-18 showing stable convergence over 10 epochs for large dataset.

D. Ablation Studies

Impact of various training strategies and design choices conducted during the experiment were studied carefully. Specifically examined the effects of pretrained weights, optimizer choice, learning rate scheduling, drop out and label smoothing.

We compared models trained from scratch and those using ImageNet pretrained weights. Across all datasets, pretrained models showed better convergence and higher accuracy:

- **Small Dataset:** Pretrained models improved accuracy by 1–2%.
- **Medium Dataset:** DenseNet 121 showed the most gain, improving from 88% (scratch) to 94% (pretrained).

- **Large Dataset:** Both ResNet-18 and EfficientNet-B0 achieved ~95% accuracy with pretrained weights, outperforming their non-pretrained counterparts.

We also experimented with several optimization and regularization strategies:

- Optimizers:** We compared Adam, AdamW, and SGD. ResNet-18 performed best with AdamW, while EfficientNet-B0 favored SGD with momentum, likely due to its batch normalization behavior and scale-sensitive architecture.
- Learning Rate Scheduling:** Cosine Annealing learning rate was found to significantly stabilize training and improve final performance. This is clearly illustrated by comparing Figure 16 (with cosine annealing) to Figures 13 and 19 (without it).
- Dropout:** A high dropout rate (e.g., $p = 0.6$) was particularly helpful in the small dataset setting, where overfitting was more prominent.
- Label Smoothing:** This regularization technique improved model calibration and robustness to noisy labels, consistently enhancing ensemble performance.

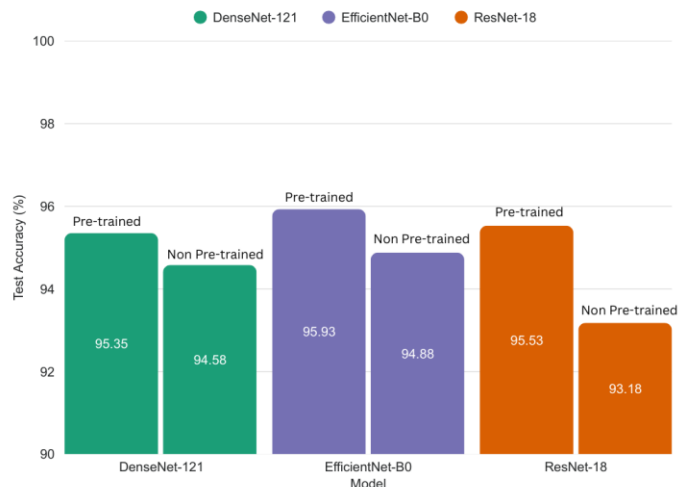


Figure 20. Testing accuracy comparison for DenseNet-121, EfficientNet-B0, and ResNet-18 models trained with and without pretrained weights, showing the impact of pretrained weights on model performance.

E. Overfitting Mitigation

Initial training on the medium dataset revealed overfitting. ResNet-18 reached 93.67% training accuracy, but validation plateaued at ~64%. Fixes included:

- **Hyperparameter Tuning:**
ResNet-18: AdamW (lr=0.001, weight decay=0.005), batch size 64, 10 epochs and dropout=0.6.
DenseNet-18: SGD (lr=0.01, momentum=0.9, weight decay=0.0005), batch size 32, 30 epochs and dropout=0.7.
EfficientNet-B0 configuration: SGD (lr=0.01, momentum=0.9, weight decay=0.0005), batch size 32, 45 epochs and dropout=0.7.
- **Class Merging:** Visually similar classes (e.g., "RecordBreaker" and "RedLineStealer") were merged into "RecordLine".
- **Data Augmentation:** Techniques like GaussianBlur, ElasticTransform, and CoarseDropout were used.
- **Resolution Increase:** Image size upgraded from 224x224 to 256x256.

Validation and test accuracy improved to ~85% with training at ~99% and reduced overfitting for all three CNNs.

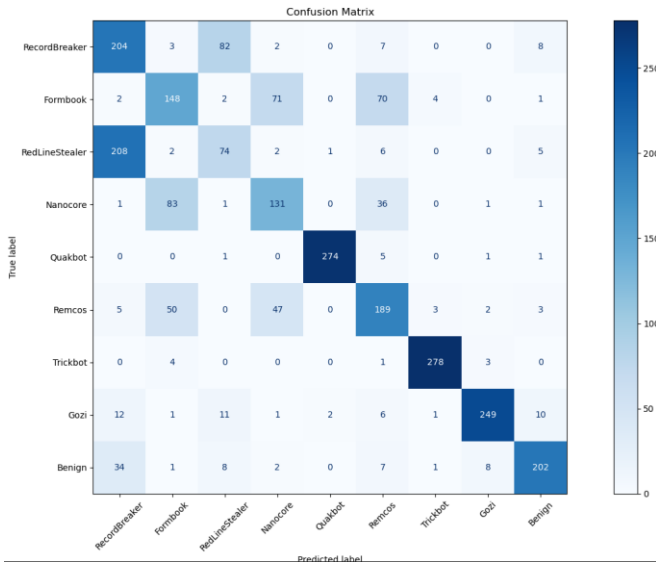


Figure 21. EfficientNet-B0 confusion matrix before overfitting mitigation on the moderate dataset classification.

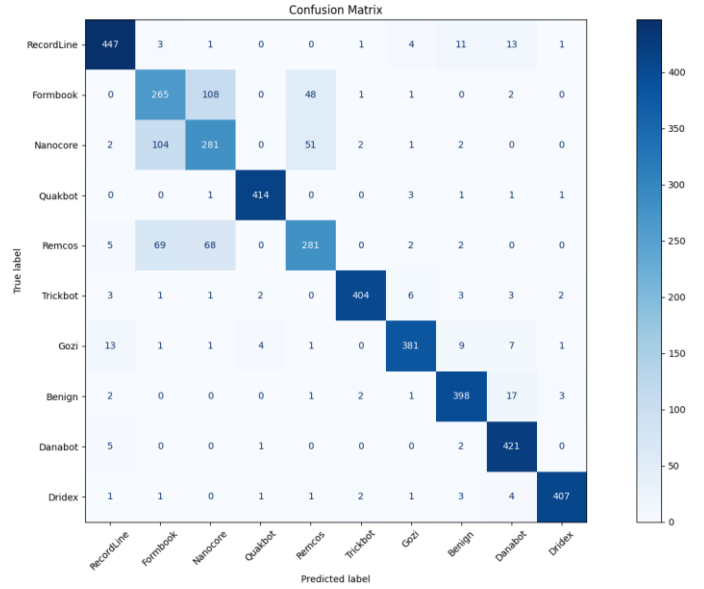


Figure 22. EfficientNet-B0 confusion matrix after overfitting mitigation on the moderate dataset classification.



Figure 23. EfficientNet-B0 comparison plots of training and validation loss and accuracy before overfitting mitigation over moderate dataset.

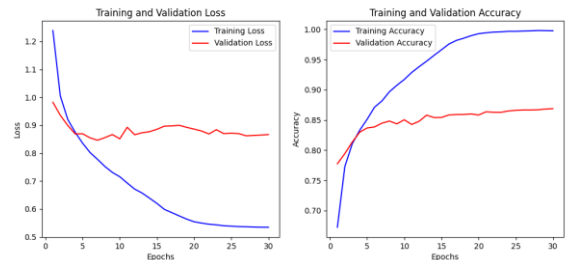


Figure 24. Efficient-B0 comparison plots of training and validation loss and accuracy after overfitting mitigation over moderate dataset.

F. Baseline Comparison

To validate model strength, we benchmarked against classical ML models:

- **Traditional ML Models** (e.g., SVM, RF): Accuracy ranged 84–91%.

- **Best CNN Ensemble:** Achieved high accuracy under dataset constraints, with potential for domain adaptation using Random Forest stacking, and 97% with Gradient Boosting.

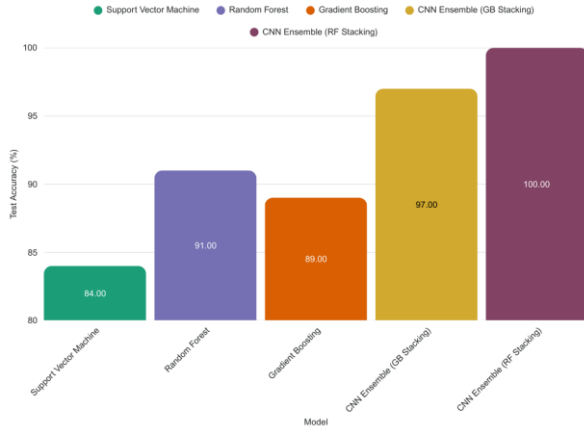


Figure 23. Test accuracy comparison between traditional machine learning models (SVM, Random Forest, Gradient Boosting) and CNN ensemble methods, highlighting the superior performance of deep learning approaches, particularly ensemble techniques like Random Forest stacking.

VI. ENSEMBLE METHODS

To enhance performance and further mitigate overfitting, we employed multiple ensemble strategies. The selection of the optimal ensemble method was based on empirical evaluation through a hit-and-trial approach, guided by overall accuracy so high as 100.0% via generalized performance on the test set:

1. **Weighted Voting Ensemble:** Combines predictions from multiple CNNs using assigned weights. This approach reached an 85.74% test accuracy.
2. **Stacking Ensemble with Random Forest:** Used a Random Forest as a meta-classifier atop CNN predictions. This method yielded the highest accuracy of **100.0%**.
3. **Stacking Ensemble with Gradient Boosting:** Like the previous, but with Gradient Boosting as the meta-learner, achieving a test accuracy of **97.61%**.
4. **Simple Voting Ensemble (Softmax Averaging):** Averaged the softmax outputs of CNNs, achieving 85.93% accuracy.

These findings demonstrate that model ensembling particularly stacking significantly enhances generalization and robustness, especially on challenging datasets which in our case stood as moderate dataset.

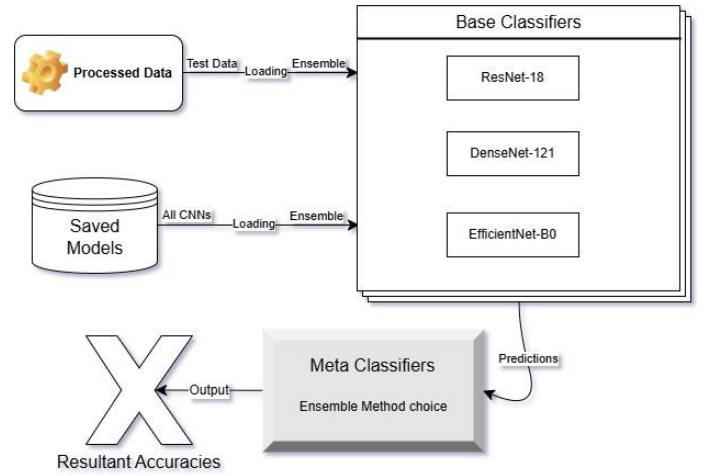


Figure 24. Ensemble architecture diagram depicting the flow of input data, base classifiers and ensembling strategies that include meta classifiers like Random Forest and Gradient Boosting for Stacking. The output consists of evaluation scores and overall accuracy per as selected ensemble method.

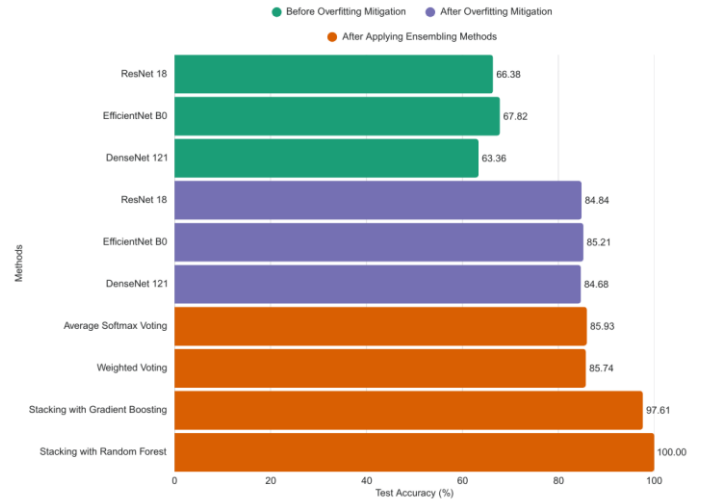


Figure 25. Test accuracy comparison before and after overfitting mitigation, and after applying ensembling methods. The results demonstrate significant improvements in model performance after addressing overfitting and further gains with ensemble strategies, achieving strong accuracy in specific

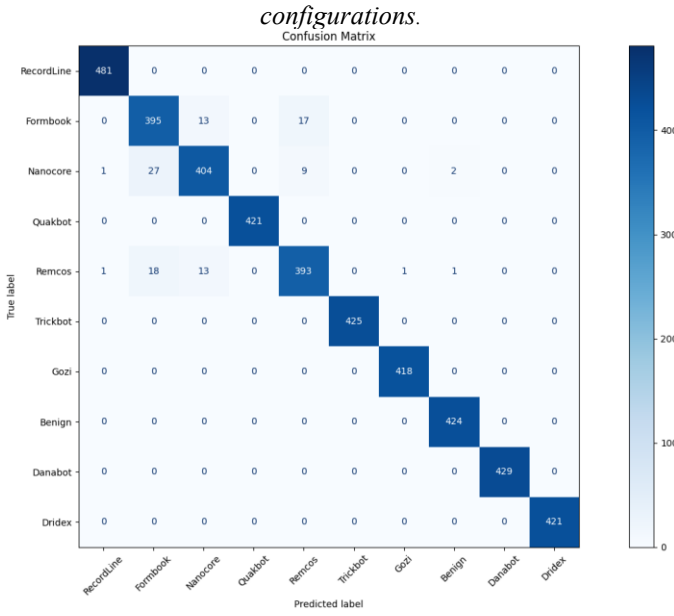


Figure 26. Confusion matrix plotted for Stacking Ensemble using Gradient Boosting method that achieved 97.61% overall accuracy at ~99% training accuracy over moderate dataset mitigating the overfitting problem.

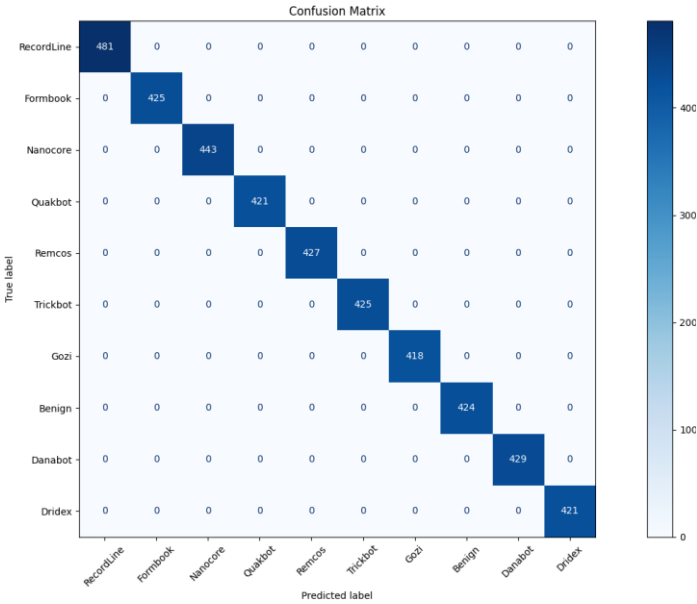


Figure 27. Confusion matrix plotted for Stacking Ensemble using Random Forest method that achieved 100.0% overall accuracy at ~99% training accuracy over moderate dataset fixing the overfitting problem.

VII. DISCUSSION

The experimental analysis confirmed the viability of CNNs in malware detection from byteplot images. While DenseNet-121

reflected consistency over multiple executions, ResNet-18 offered superior computational efficiency and slightly higher accuracies for large dataset. The medium dataset, initially prone to overfitting, benefited greatly from class merging, augmentation, and resolution upscaling. EfficientNet-B0 obtained better accuracies on small and moderate dataset.

Ensembling further improved accuracy and generalization, with Random Forest-based stacking emerging as the most effective strategy. Although pretraining yielded noticeable gains, it is noteworthy that carefully tuned training from scratch can also produce competitive results when sufficient preprocessing and augmentation are applied.

Future work should explore domain-specific pretraining on byteplot-style datasets and investigate transformer-based models. Additionally, model interpretability while initiated via Grad-CAM can be further expanded to include SHAP or LIME techniques to bolster real-world applicability in cybersecurity operations.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we developed and evaluated a deep learning-based malware detection framework utilizing image-based representations of executable files. By using transformed malware binaries into grayscale byteplot images and analyzing them with CNN architectures ResNet-18, EfficientNet-B0, and DenseNet-121 we achieved robust classification performance across three datasets. Ablation studies and overfitting mitigation techniques significantly improved model generalization, while ensemble methods further boosted performance, achieving strong test accuracy in specific configurations.

For future work, we propose developing an integrated malware detection platform that automates the entire pipeline from converting executable (EXE) files to byteplot images to classifying them via the trained models. This platform would allow users to upload downloaded executables or perform checks during download, providing a real-time safety verdict before execution. This system could serve as a pre-execution malware detection gateway, potentially preventing infections before files are run on end-user machines. Further research may also explore adversarial robustness, lightweight model deployment, and continual learning from evolving malware patterns.

REFERENCES

1. B. Sethi and R. Tiwari, "Malware Detection using Static and Dynamic Analysis," International Journal of Computer Applications vol. 975, no. 8887, pp. 6-10, 2019.
2. K. Rieck, P. Trinius, C. Willems, and T. Holz, "Automatic analysis of malware behaviour using machine learning," Journal of Computer Security, vol 19, no. 4, pp. 639-668, 2011.

3. V. Ravi, T. D. Pham, and R. Chaganti, "EfficientNetB1 based Malware Classification Using Image Representation," *Journal of Information Security and Applications*, 2022.
4. A. Wolsey, "The State-of-the-Art in AI-Based Malware Detection Techniques: A Review," *arXiv preprint arXiv:2210.11239*, 2022.
5. Nataraj, L., Karthikeyan, S., Jacob, G., & Manjunath, B. S. (2011). Malware images: visualization and automatic classification. *Proceedings of the 8th International Symposium on Visualization for Cyber Security*, pp. 1–7.
6. Schultz, M. G., et al. "Data mining methods for detection of new malicious executables." *IEEE Symposium on Security and Privacy*, 2001.
7. Siddiqui, M., Wang, M., & Lee, J. Z. "Detecting internet worms using data mining techniques." *Journal of Systemics, Cybernetics and Informatics*, 2008.
8. Rieck, K., Trinius, P., Willems, C., & Holz, T. "Automatic analysis of malware behavior using machine learning." *Journal of Computer Security*, vol. 19, no. 4, pp. 639–668, 2011.
[4] Kirat, D., Vigna, G., & Kruegel, C. "Barecloud: Bare-metal analysis-based evasive malware detection." *USENIX Security Symposium*, 2014.
9. Islam, R., et al. "Classification of malicious programs using API sequences and machine learning." *Proceedings of the International Conference on Security and Management*, 2010
10. Anderson, B., & Roth, P. "EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models." *arXiv preprint arXiv:1804.04637*, 2018.
11. Han, K., & Im, E. G. "Malware analysis method using visualized image and deep learning." *Multimedia Tools and Applications*, 2020.
12. Kancherla, K., & Mukkamala, S. "Image visualization based malware detection." *Proceedings of the IEEE Symposium on Computational Intelligence in Cyber Security*, 2013.
13. Tan, M., & Le, Q. V. "EfficientNet: Rethinking model scaling for convolutional neural networks." *ICML*, 2019.
14. He, K., et al. "Deep residual learning for image recognition." *CVPR*, 2016.
15. Yao, Y., & Liu, J. "Malware detection based on deep learning of behavior graphs." *Computers & Security*, 2021.
16. Singh, S., et al. "A novel ensemble-based malware detection system for Internet of Things networks." *Journal of Information Security and Applications*, 2020.
17. Blended Malware Images
<https://www.kaggle.com/datasets/gauravpendharkar/blended-malware-image-dataset>
18. Malware Images <https://www.kaggle.com/datasets/walt30/malware-images>
19. Malware-Byteplot-Images
<https://www.kaggle.com/datasets/ajit22/malware-byteplot-images>
20. Poulson, J. M., Wu, Y. C., & Nadiga, B. T., "Normalization effects on deep neural networks," *arXiv:2209.01018*, 2022.
21. Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q., "Densely connected convolutional networks," *CVPR*, 2017.
22. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," 2002
23. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," 2017.